21 Working with json and schema evolution

JSON File

```
df = spark.read.json("/data/orders_nested.json")
df.printSchema()
df.show(truncate=False)

df.select("customer.customer_id","customer.name", "items",
"order_date", "order_id").show(truncate=False)

# use exploded to convert list into rows
df_exploded = df.withColumn("item", explode("items"))
df_exploded.select("customer.customer_id", "item.price").sh
ow(truncate=False)

# pass schema of your json (best practice)
orders_schema = StructType([
    StructField("order_id", IntegerType()),
    StructField("customer", StructType([
        StructField("id", IntegerType()),
        StructField("name", StringType())
    ])),
    StructField("items", ArrayType(
        StructType([
            StructField("product", StringType()),
            StructField("price", IntegerType())
        ])
    ))
])

df = spark.read.schema(orders_schema).json("/data/orders_ne
```

```
sted.json")
df.printSchema()
```

Schema Evolution

What is schema evolution

- Schema evolution means **data schema changes over time** while old data still exists
- Common changes:
  - Add column
  - Drop column
  - Rename column
  - Change data type

What is mergeSchema

- `mergeSchema=true` tells Spark to **merge schemas of all Parquet ORC files at read time**
- Spark computes **union of columns** across files

Where it works

- ✅ Parquet
- ✅ ORC
- ✅ Delta Tables
- ❌ CSV
- ❌ JSON

Reason

Only Parquet , ORC store schema in file metadata.

Default behavior (without mergeSchema)

- Spark reads schema from **one file only**
- Which file is chosen is **not guaranteed**
- Other files are read using that schema
- Missing columns → `null`
- Extra columns may be silently ignored

With mergeSchema enabled

```
spark.read.option("mergeSchema", "true").parquet("/data")
```

Spark:

- reads schema from **all files**
- merges columns
- resolves compatible types
- missing columns → `null`

What mergeSchema can handle

- Adding new columns ✅
- Missing columns ✅
- Column order differences ✅

What mergeSchema cannot handle well

- Column rename ❌ (seen as drop + add)
- Conflicting data types ❌
- Type narrowing ❌

Performance impact

- Spark must read metadata from **every file**

- Slower job startup
- Expensive for folders with many small files

That's why it is **disabled by default**.

Best practices

- Prefer consistent schema at write time
- Avoid frequent schema changes
- Use mergeSchema only when required
- Do not rely on Spark's default schema picking

One line takeaway

> mergeSchema forces Spark to union schemas across Parquet files, trading startup performance for correctness.

Interview ready answer

> mergeSchema enables Spark to merge schemas across Parquet or ORC files at read time, but it is expensive and should be used cautiously.

```
# create file
orders_schema = 'order_id int,customer_id int,product_id in
t,unit_price float,order_date date,order_status string,stat
e string,quantity int'
df = spark.read.format("csv").schema(orders_schema).option
("header" , "true").load("/data/orders_300mb.csv")
df.write.format("parquet").mode("overwrite").save("/data/or
ders_parquet/")

# apped data with additional column
df_new = df.filter(col("order_id")<=10).withColumn("sales",
col("unit_price")*col("quantity"))
df_new.write.format("parquet").mode("append").save("/data/o
rders_parquet/")
```

```
df_p = spark.read.parquet("/data/orders_parquet/")
df_p.printSchema()
df_p.filter(col("order_id")==2).show()

# with merge schema option
df_p = spark.read.parquet("/data/orders_parquet/" , mergeSc
hema = True)
```